ShopFlow Auth Service

Generate a JWT Secret with Python and Configure It in IntelliJ IDEA

This guide shows how to generate a strong HS256 signing secret on Windows, configure it as an environment variable in IntelliJ IDEA, and make it available to Spring Boot without hardcoding it in application.properties or Java source code.

1. Check that Python is installed

Open Windows PowerShell or the IntelliJ terminal and run:

```
python --version
```

If that command is not recognized, try:

```
py --version
```

Continue with whichever command prints a Python version. If neither works, fix or install Python first.

2. Generate the secret

If python works, run this exact PowerShell command:

```
python -c "import secrets,base64; print(base64.b64encode(secrets.token_bytes(32)).decode())
```

If only py works, use:

```
py -c "import secrets,base64; print(base64.b64encode(secrets.token_bytes(32)).decode())
```

The output is a Base64-encoded value representing 32 cryptographically random bytes (256 bits), suitable for an HS256 JWT signing key. Generate it once for this local environment and keep it private.

3. Add it to IntelliJ IDEA

1	Open Run → Edit Configurations.
2	Select AuthServiceApplication.
3	Find Environment variables and click the edit icon.
4	Click + to add a row. In the Name column enter JWT_SECRET.
5	In the Value column paste the complete generated Base64 value. If needed, double-click the Value cell or press Tab from Name to Value.
6	Click OK in the environment-variable editor, then Apply and OK in the run configuration.

4. Connect the variable to Spring Boot

In src/main/resources/application.properties, add:

```
jwt.secret=${JWT_SECRET}
```

Do not put the actual secret value in application.properties, Java code, Git, screenshots, or chat. Spring Boot resolves ${JWT_SECRET} from the environment supplied by IntelliJ.

5. Restart and verify

• Stop the currently running AuthServiceApplication.

• Start it again from IntelliJ using the updated run configuration.

• If it starts, the environment variable is available to the process. If a missing-variable error appears, the configuration was not passed correctly.

• Do not print JWT_SECRET in logs to verify it.

6. Security notes

• Never share the secret or commit it to GitHub.

• Use different secrets for local development, testing, and production.

• Keep the same secret across application restarts and across instances that must validate the same tokens.

• If a secret is exposed, generate a new one and plan for tokens signed with the old key to stop validating.

• For production, use a platform secret manager such as AWS Secrets Manager rather than embedding secrets in deployment files.

Next step: create JwtService to read the configured secret, sign tokens using HS256, include the user's email and role, and set an expiration time.